

Vedant Vartak

DATA 260 - Agentic AI and Distributed Systems

Homework 5: Tools, Reliability, and a Safe Local Agent

GitHub: <https://github.com/vedantvartakwork/data260-3539>

Verified implementation commit: df1234ae0692f597e2b430a73202883abf20ddf3

Resolved submission tag: hw5 -> df1234ae0692f597e2b430a73202883abf20ddf3

Item	Value	Item	Value
SID4	3539	PORT_BASE	8839
PREFIX	s3539	SEED	3539
VERIFY_SEED	263539	DOMAIN_ID	3
Domain	Grocery supply and recall notices	Local model	qwen3:8b
Hardware	Apple M4 MacBook Air	Memory / CPU	16 GB / 10 cores
Platform	macOS 15.3	Ports	API 8839 / UI 5173

This cumulative submission extends the authenticated FastAPI/MySQL application with a manufacturer relationship, Redux Toolkit and Axios, two MCP servers, strict tool schemas, bounded retries, and a local Ollama tool-calling loop with deterministic safety enforcement.

Submission contents

The repository includes RUN_LOG.txt, METRICS.md, REFLECTION.md, TOOL_CONTRACTS.md, AI_USE.md, verification.json, 150 raw fault-injection rows, MCP/agent/API raw outputs, Postman collection and environment files, and this screenshot-backed report.

1. Completion and verification

The final cumulative checks passed without exceptions or skipped requirements.

Area	Evidence	Result
Database/API	12 manufacturers; 500 recalls; 200 events; 14/14 authenticated checks	PASS
React/Redux	Home, create, update, delete; 10-page access to all 500 records	PASS
MealDB MCP	Exactly four live tools demonstrated	PASS
Domain MCP	Exactly search/detail/aggregate; valid and invalid calls	PASS
Reliability	150 deterministic calls; 3-attempt bounded retry	PASS
Offline tests	9/9 tool and safety tests	PASS
Cumulative suite	34/34 Python tests; React production build	PASS
Local agent	Four qwen3:8b scenarios; one enforced safety block	PASS
Verifier	12/12 checks	PASS

The verifier independently inventories both MCP servers, calls one tool from each, checks the live API health endpoint, counts all 150 experiment rows, runs the offline runner and 34-test suite, and rebuilds the React client.

2. Manufacturer CRUD request/response evidence

Every operation is labeled with the exact HTTP request, returned status, and response excerpt.

LIVE API REQUEST / RESPONSE EVIDENCE

create_manufacturer

REQUEST: POST /api/v1/manufacturers
STATUS: 201 (expected 201)
RESPONSE: {"id": 13}

list_manufacturers

REQUEST: GET /api/v1/manufacturers?limit=50
STATUS: 200 (expected 200)
RESPONSE: {"returned_records": 13, "first": {"id": 1, "name": "Grocery Safety Manufacturer 01",
"contact_name": "Safety Contact 01", "contact_email": "safety01@example.edu", "created_at":
"2026-10-04T21:27:53", "updated_at": "2026-10-04T21:27:53"}}

read_manufacturer

REQUEST: GET /api/v1/manufacturers/{id}
STATUS: 200 (expected 200)
RESPONSE: {"id": 13}

update_manufacturer

REQUEST: PUT /api/v1/manufacturers/{id}
STATUS: 200 (expected 200)
RESPONSE: {"id": 13}

Manufacturer create, list, read, and update all returned their expected statuses.

3. Recall create, list, read, and relationship evidence

The previously unclear recall operations are now all shown explicitly, including filtered listing and relationship lookup.

LIVE API REQUEST / RESPONSE EVIDENCE

create_recall

REQUEST: POST /api/v1/recalls
STATUS: 201 (expected 201)
RESPONSE: {"id": 501, "recall_code": "REC-3539-INT01", "product_name": "Integration Recall Evidence"}

list_recalls

REQUEST: GET /api/v1/recalls?q=Integration+Recall&page=1&page_size=50
STATUS: 200 (expected 200)
RESPONSE: {"total": 1, "page": 1, "page_size": 50, "returned_records": 1, "first_recall_code": "REC-3539-INT01"}

read_recall

REQUEST: GET /api/v1/recalls/{id}
STATUS: 200 (expected 200)
RESPONSE: {"id": 501, "recall_code": "REC-3539-INT01", "product_name": "Integration Recall Evidence"}

relationship_endpoint

REQUEST: GET /api/v1/recalls/by-manufacturer/{manufacturer_id}
STATUS: 200 (expected 200)
RESPONSE: {"returned_records": 1, "first": {"product_name": "Integration Recall Evidence", "recall_code": "REC-3539-INT01", "units_affected": 3539, "manufacturer_id": 13, "brand_name": "Integration Foods 3539 Updated", "submitter_email": "student@example.edu", "category": "Packaged Foods", "recall_details": "Selected packages may contain an undeclared allergen and should be returned for a refund.", "terms_accepted": true, "id": 501, "created_at": "2026-10-04T21:29:22", "updated_at": "2026-10-04T21:29:22", "manufacturer": {"id": 13, "name": "Integration Foods 3539 Updated", "contact_name": "Vedant Vartak", "contact_email": "integration3539@example.edu", "created_at": "2026-10-04T21:29:22", "updated_at": "2026-10-04T21:29:22"}, "events": []}}

Recall create/list/read and GET by manufacturer all returned the created record.

4. Recall update, delete, FK block, and cleanup

The write half proves update/delete behavior, the 404 after deletion, and RESTRICT enforcement.

LIVE API REQUEST / RESPONSE EVIDENCE

delete_referenced_manufacturer_blocked

REQUEST: DELETE /api/v1/manufacturers/{id}
STATUS: 409 (expected 409)
RESPONSE: {"detail": "Cannot delete a manufacturer that still has recall notices"}

update_recall

REQUEST: PUT /api/v1/recalls/{id}
STATUS: 200 (expected 200)
RESPONSE: {"id": 501, "recall_code": "REC-3539-INT01", "product_name": "Integration Recall Evidence Updated"}

delete_recall

REQUEST: DELETE /api/v1/recalls/{id}
STATUS: 204 (expected 204)
RESPONSE: null

deleted_recall_is_404

REQUEST: GET /api/v1/recalls/{id}
STATUS: 404 (expected 404)
RESPONSE: {"detail": "Recall notice not found"}

delete_manufacturer

REQUEST: DELETE /api/v1/manufacturers/{id}
STATUS: 204 (expected 204)
RESPONSE: null

All status codes match the recorded 14/14 live integration checks.

5. Database schema plus implementation code

The live MySQL view and the immediately adjacent implementation/output panel establish both schema and behavior.

```
Last login: Sun Oct 4 00:34:07 on tty8818
cd /Users/vedantvartak/Documents/Codes/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py
vedantvartak@Vedants-MacBook-Air ~ % cd /Users/vedantvartak/Documents/Codes/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py

VEDANT VARTAK - DATA 260 HOMEWORK 5 - MYSQL EVIDENCE
timestamp_utc: 2026-10-04T07:34:11.556960+00:00
database: k3539_rel
tables: manufacturers, recall_events, recall_notices, sessions, users

ROW COUNTS:
manufacturers: 12
recall_notices: 500
recall_events: 200
users: 1
sessions: 1

manufacturers_columns:
id | INTEGER | nullable=False
name | VARCHAR(160) | nullable=False
contact_name | VARCHAR(160) | nullable=False
contact_email | VARCHAR(255) | nullable=False
created_at | DATETIME | nullable=False
updated_at | DATETIME | nullable=False

recall_notices_columns:
id | INTEGER | nullable=False
product_name | VARCHAR(160) | nullable=False
recall_code | VARCHAR(32) | nullable=False
units_affected | INTEGER | nullable=False
manufacturer_id | INTEGER | nullable=False
brand_name | VARCHAR(160) | nullable=False
submitter_email | VARCHAR(255) | nullable=False
category | VARCHAR(60) | nullable=False
recall_details | TEXT | nullable=False
terms_accepted | TINYINT | nullable=False
created_at | DATETIME | nullable=False
updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
fk_recall_notices_manufacturer: {'manufacturer_id' -> manufacturers['id'] ondelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codes/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.txt
vedantvartak@Vedants-MacBook-Air: data260-3539 %
```

12 manufacturers, 500 recalls, 200 events, required columns, and RESTRICT foreign key.

DATABASE + API VALIDATION CODE AND OUTPUT

IMPLEMENTATION

RecallCreate validates positive units, email, exact category, accepted terms, unique recall_code, and manufacturer_id. The API catches IntegrityError and returns HTTP 409; the FK uses ON DELETE RESTRICT.

CODE

```
class RecallCreate(RecallInput): pass

if db.get(Manufacturer, payload.manufacturer_id) is None:
    raise HTTPException(404, 'Manufacturer not found')
try: db.commit()
except IntegrityError:
    db.rollback(); raise HTTPException(409, 'Recall code must be unique')
```

MEASURED OUTPUT

```
manufacturers=12; recall_notices=500; recall_events=200
FK: manufacturer_id -> manufacturers.id ON DELETE RESTRICT
14/14 authenticated live API checks passed
```

Validation/transaction code with measured database and API output.

6. Redux Home - code and UI in one screenshot

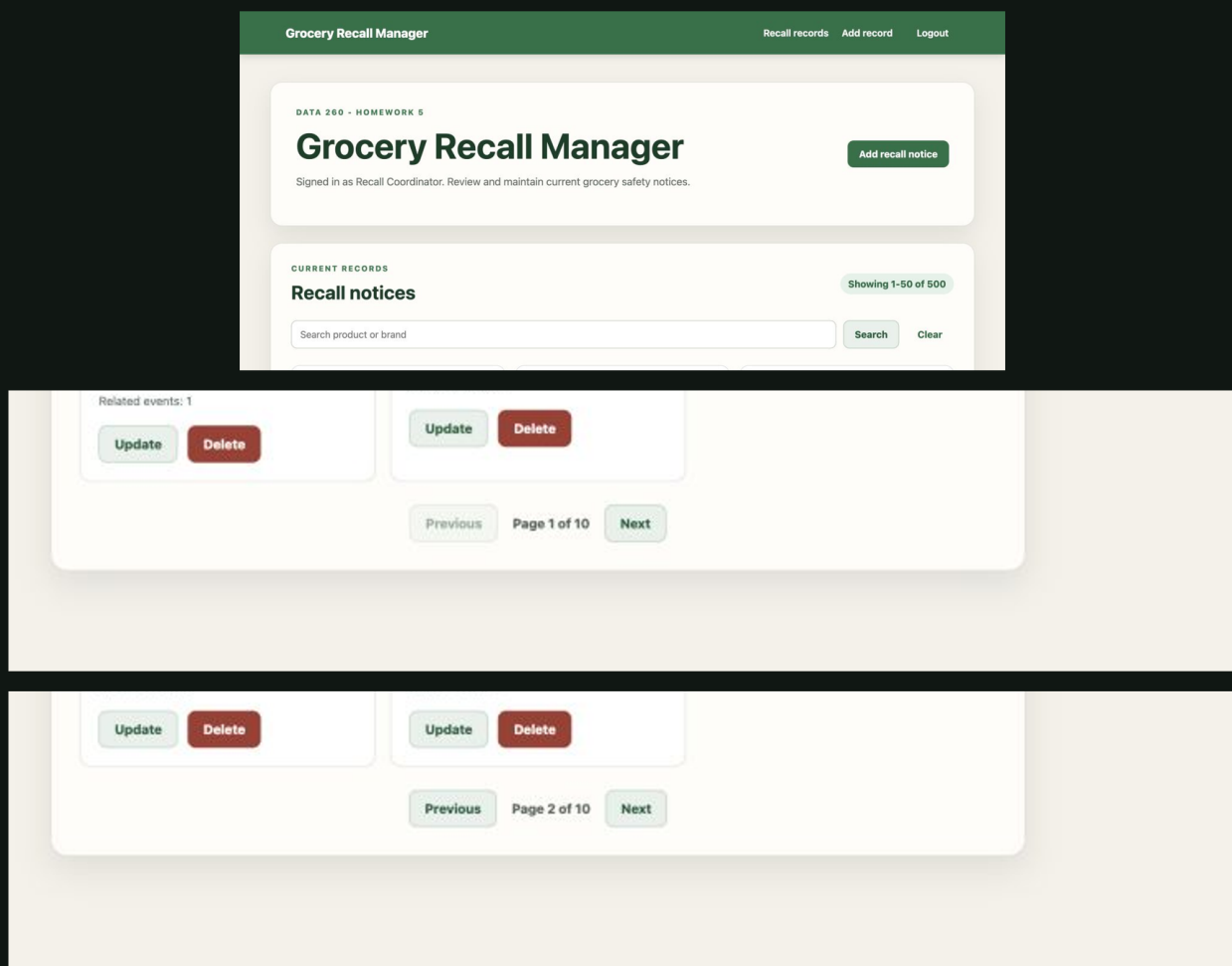
The composite contains the actual thunk/slice fields and visible UI output, including page 1 and page 2.

REDUX HOME: PAGINATED ACCESS TO ALL 500 RECORDS

RELEVANT REDUX / REACT CODE

```
export const fetchRecalls = createAsyncThunk('recalls/fetch',
  async ({ query = '', page = 1 } = {}, thunkApi) => {
    const params = { page, page_size: 50 };
    const response = await apiClient.get('/recalls', { params });
    return response.data;
  });

const { items, page, pageSize, total } = useSelector(s => s.recalls);
<span>Showing {firstShown}-{lastShown} of {total}</span>
<span>Page {page} of {totalPages}</span>
```



Showing 1-50 of 500; page 1/10 and page 2/10 prove access beyond the first 50 records.

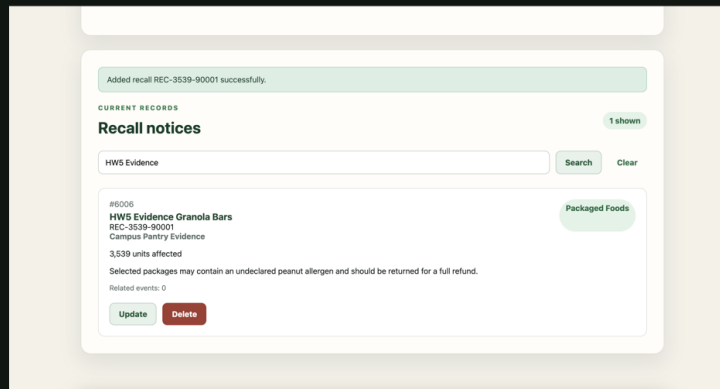
7. Redux Create - code and UI in one screenshot

The create thunk and persisted success state appear in the same captured frame.

REDUX CREATE: THUNK + SAVED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(createRecall(payload)).unwrap();
export const createRecall = createAsyncThunk('recalls/create', async (payload) => {
  const response = await apiClient.post('/recalls', payload);
  return response.data;
});
```



createRecall returned the saved database record and the UI displayed the success notice.

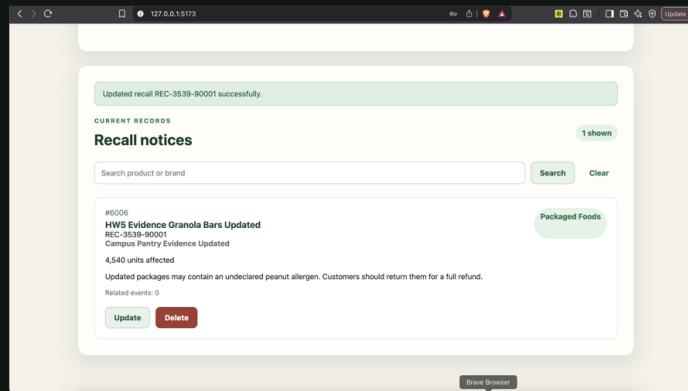
8. Redux Update - code and UI in one screenshot

The update thunk/reducer and revised record output appear together.

REDUX UPDATE: THUNK + UPDATED UI OUTPUT

RELEVANT REDUX / REACT CODE

```
await dispatch(updateRecall({ id: Number(id), payload })).unwrap();
.addCase(updateRecall.fulfilled, (state, action) => {
  const index = state.items.findIndex(item => item.id === action.payload.id);
  if (index >= 0) state.items[index] = action.payload;
});
```



The Redux item was replaced only after the API returned the updated record.

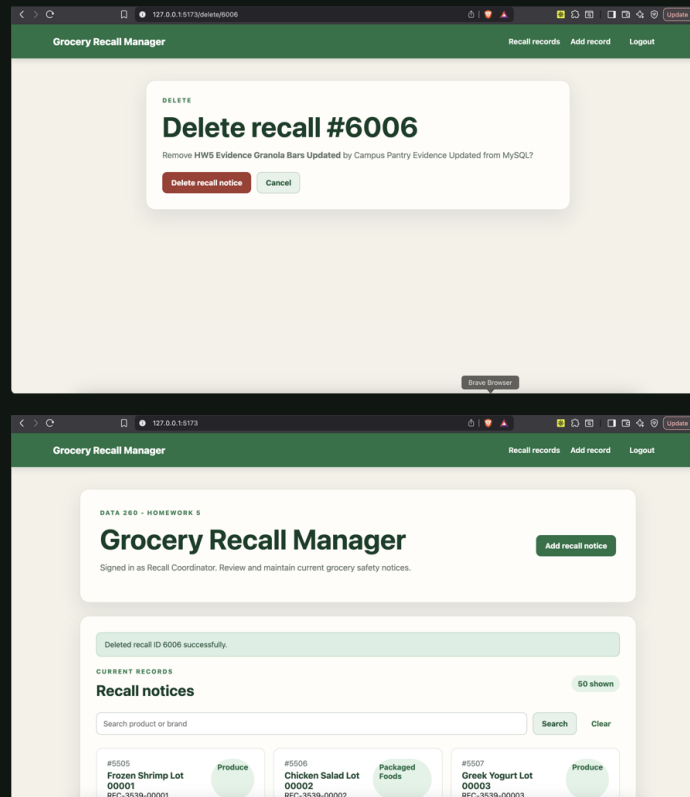
9. Redux Delete - code, confirmation, and output

The dedicated confirmation route and post-delete state appear with the reducer code in one composite screenshot.

REDUX DELETE: CONFIRMATION ROUTE + UI OUTPUT

RELEVANT REDUX / REACT CODE

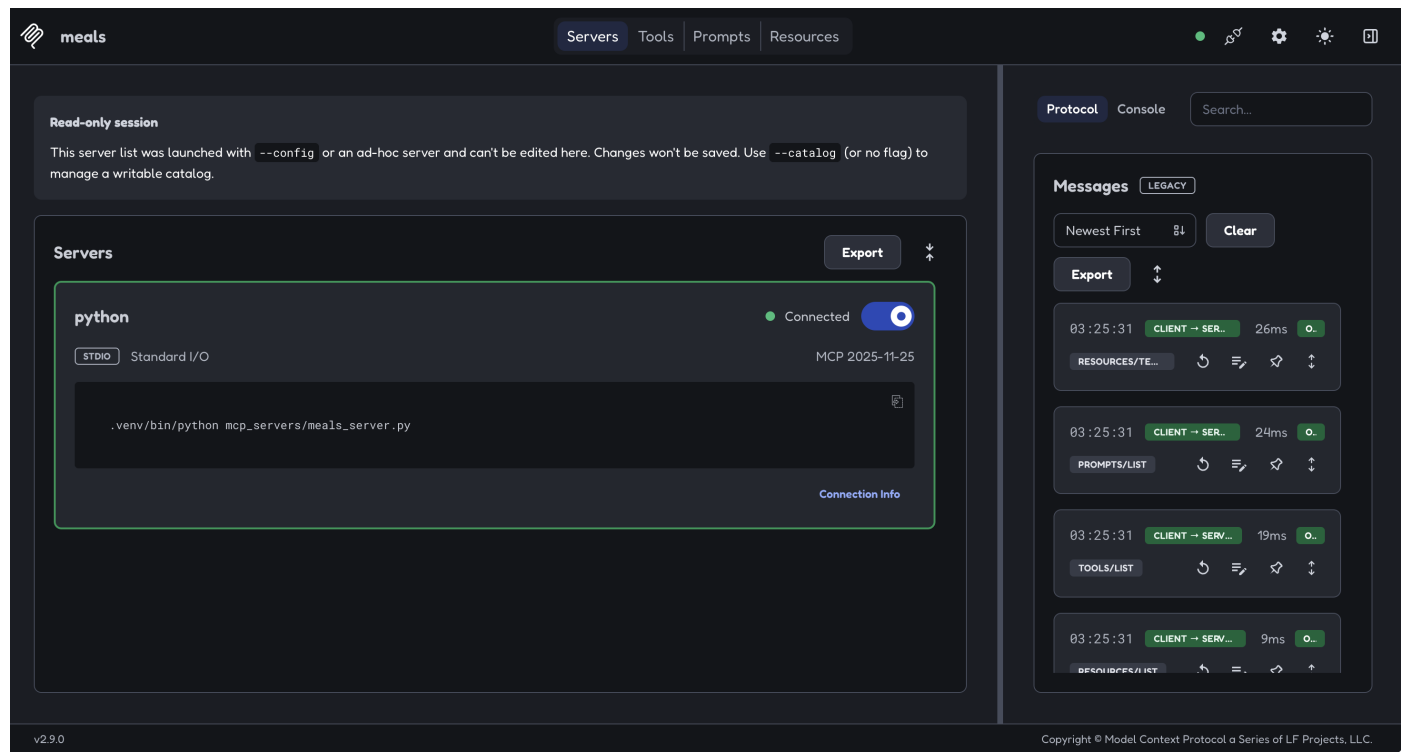
```
<Link to={`/delete/${record.id}`}>Delete</Link>
await dispatch(deleteRecall(Number(id))).unwrap();
.addCase(deleteRecall.fulfilled, (state, action) => {
  state.items = state.items.filter(item => item.id !== action.payload);
  state.notice = `Deleted recall ID ${action.payload} successfully.`;
});
```



Explicit confirmation precedes deletion; the success notice follows API completion.

10. MealDB MCP server connection

The Inspector connected over stdio using the project virtual environment and exposed exactly four required tools.



Connected MealDB MCP server in Inspector; tools, prompts, and resources inventories completed successfully.

11. MealDB MCP - search_meals_by_name

The exact Inspector input JSON and structured output are readable in the evidence panel.

MEALDB MCP TOOL 1/4: search_meals_by_name

INPUT JSON

```
{
  "query": "Arrabiata",
  "limit": 5
}
```

STRUCTURED OUTPUT

```
{
  "meals": [
    {
      "id": "52771",
      "name": "Spicy Arrabiata Penne",
      "area": "Italian",
      "category": "Vegetarian",
      "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"
    }
  ],
  "message": null
}
```

Input and output for search_meals_by_name; complete raw output is retained in mcp_tool_outputs.json.

12. MealDB MCP - meals_by_ingredient

The exact Inspector input JSON and structured output are readable in the evidence panel.

MEALDB MCP TOOL 2/4: meals_by_ingredient

INPUT JSON

```
{
  "ingredient": "chicken",
  "limit": 5
}
```

STRUCTURED OUTPUT

```
{
  "meals": [
    {
      "id": "52940",
      "name": "Brown Stew Chicken",
      "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
    },
    {
      "id": "53161",
      "name": "Chicken & chorizo rice pot",
      "thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763280767.jpg"
    },
    {
      "id": "52846",
      "name": "Chicken & mushroom Hotpot",
      "thumb": "https://www.themealdb.com/images/media/meals/uuuspp1511297945.jpg"
    },
    {
      "id": "52796",
      "name": "Chicken Alfredo Primavera",
      "thumb": "https://www.themealdb.com/images/media/meals/syqypv1486981727.jpg"
    },
    {
      "id": "52934",
      "name": "Chicken Basquaise",
      "thumb": "https://www.themealdb.com/images/media/meals/wruvqv1511880994.jpg"
    }
  ],
  "message": null
}
```

Input and output for meals_by_ingredient; complete raw output is retained in mcp_tool_outputs.json.

13. MealDB MCP - meal_details

The exact Inspector input JSON and structured output are readable in the evidence panel.

MEALDB MCP TOOL 3/4: meal_details

INPUT JSON

```
{
  "id": "52771"
}
```

STRUCTURED OUTPUT

```
{
  "id": "52771",
  "name": "Spicy Arrabiata Penne",
  "category": "Vegetarian",
  "area": "Italian",
  "instructions": "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes.\n\nIn a large skillet over medium-high heat, add the olive oil and heat until the oil starts to shimmer. Add the garlic and cook, stirring, until fragrant, 1 to 2 minutes. Add the chopped tomatoes, red chile flakes, Italian seasoning and salt and pepper to taste. Bring to a boil and cook for 5 minutes. Remove from the heat and add the chopped basil.\n\nDrain the pasta and add it to the sauce. Garnish with Parmigiano-Reggiano flakes and more basil and serve warm.",
  "image": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
  "source": null,
  "youtube": "https://www.youtube.com/watch?v=1IszT_gul08",
  "ingredients": [
    {
      "name": "penne rigate",
      "measure": "1 pound"
    },
    {
      "name": "olive oil",
      "measure": "1/4 cup"
    },
    {
      "name": "garlic",
      "measure": "3 cloves"
    },
    {
      "name": "chopped tomatoes",
      "measure": "1 tin"
    },
    {
      "name": "red chilli flakes",
      "measure": "1/2 teaspoon"
    },
    {
      "name": "Italian seasoning",
      "measure": "1/2 teaspoon"
    },
    {
      "name": "basil",
      "measure": "6 leaves"
    },
    {
      "name": "Parmigiano-Reggiano",
      "measure": "sprinkling"
    }
  ]
}
```

Input and output for meal_details; complete raw output is retained in mcp_tool_outputs.json.

14. MealDB MCP - random_meal

The exact Inspector input JSON and structured output are readable in the evidence panel.

MEALDB MCP TOOL 4/4: random_meal

INPUT JSON

```
{}
```

STRUCTURED OUTPUT

```
{
  "id": "53037",
  "name": "Coddled pork with cider",
  "category": "Pork",
  "area": "Irish",
  "instructions": "STEP 1\nHeat the butter in a casserole dish until sizzling, then fry the pork for 2-3 mins on each side until browned. Remove from the pan.\nSTEP 2\nTip the bacon, carrot, potatoes and swede into the pan, then gently fry until slightly coloured. Stir in the cabbage, sit the chops back on top, add the bay leaf, then pour over the cider and stock. Cover the pan, then leave everything to gently simmer for 20 mins until the pork is cooked through and the vegetables are tender.\nSTEP 3\nServe at the table spooned straight from the dish.",
  "image": "https://www.themealdb.com/images/media/meals/7vpsfp1608588991.jpg",
  "source": "https://www.bbcgoodfood.com/recipes/irish-coddled-pork-cider",
  "youtube": "https://www.youtube.com/watch?v=ftn_4mc-65o",
  "ingredients": [
    {
      "name": "Butter",
      "measure": "Knob"
    },
    {
      "name": "Pork Chops",
      "measure": "2"
    },
    {
      "name": "Bacon",
      "measure": "4"
    },
    {
      "name": "Potatoes",
      "measure": "2"
    },
    {
      "name": "Carrots",
      "measure": "1"
    },
    {
      "name": "Swede",
      "measure": "1/2"
    },
    {
      "name": "Cabbage",
      "measure": "1/2"
    },
    {
      "name": "Bay Leaf",
      "measure": "1"
    },
    {
      "name": "Cider",
      "measure": "100ml"
    },
    {
      "name": "Chicken Stock",
      "measure": "100g"
    }
  ]
}
```

Input and output for random_meal; complete raw output is retained in mcp_tool_outputs.json.

15. Domain MCP contract - search

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 1/3: search

EXPECTED JSON INPUT SCHEMA

```
("query": "string, 2-80 characters", "limit": "integer, 1-25")
```

VALID INPUT

```
{
  "query": "shrimp",
  "limit": 1
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "matches": [
      {
        "id": 5505,
        "recall_code": "REC-3539-00001",
        "product_name": "Frozen Shrimp Lot 00001",
        "brand_name": "Grocery Safety Manufacturer 10",
        "category": "Produce",
        "units_affected": 6535
      }
    ],
    "count": 1
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "query": "s",
  "limit": 5
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid search input: query: String should have at least 2 characters"
}
```

WHY REJECTED

One character is below the declared minimum and creates an excessively broad lookup.

The search contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

16. Domain MCP contract - detail

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 2/3: detail

EXPECTED JSON INPUT SCHEMA

```
{"recall_id": "positive integer"}
```

VALID INPUT

```
{
  "recall_id": 5505
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "id": 5505,
    "recall_code": "REC-3539-00001",
    "product_name": "Frozen Shrimp Lot 00001",
    "brand_name": "Grocery Safety Manufacturer 10",
    "category": "Produce",
    "units_affected": 6535,
    "recall_details": "Grocery Safety Manufacturer 10 Frozen Shrimp lot 00001 may contain an undeclared allergen and should be returned for a refund. Consumers should follow the recall instructions.",
    "manufacturer": {
      "id": 24,
      "name": "Grocery Safety Manufacturer 10",
      "contact_name": "Safety Contact 10"
    }
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "recall_id": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid detail input: recall_id: Input should be greater than 0"
}
```

WHY REJECTED

Database IDs begin at 1, so zero is outside the accepted domain.

The detail contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

17. Domain MCP contract - aggregate

Expected JSON schema, valid input/output, rejected JSON, complete error, and rejection reason are shown together.

DOMAIN MCP CONTRACT 3/3: aggregate

EXPECTED JSON INPUT SCHEMA

```
{"group_by": "category|manufacturer", "min_units": "integer, 0-100000000"}
```

VALID INPUT

```
{
  "group_by": "category",
  "min_units": 0
}
```

VALID OUTPUT

```
{
  "ok": true,
  "data": {
    "group_by": "category",
    "groups": [
      {
        "label": "Dairy and Refrigerated",
        "recall_count": 127,
        "total_units": 1198565
      },
      {
        "label": "Meat and Seafood",
        "recall_count": 130,
        "total_units": 1327145
      },
      {
        "label": "Packaged Foods",
        "recall_count": 131,
        "total_units": 1389038
      },
      {
        "label": "Produce",
        "recall_count": 112,
        "total_units": 1098442
      }
    ]
  },
  "error": null
}
```

REJECTED JSON OBJECT

```
{
  "group_by": "submitter",
  "min_units": 0
}
```

COMPLETE RETURNED ERROR

```
{
  "ok": false,
  "data": null,
  "error": "Invalid aggregate input: group_by: String should match pattern \"^(category|manufacturer)$\""
}
```

WHY REJECTED

submitter is not an approved grouping dimension and could expose personal contact data.

The aggregate contract returns the stable {ok, data, error} envelope for both valid and invalid calls.

18. Retry code and measured behavior

The bounded implementation is shown immediately above its measured 150-call output.

RETRY IMPLEMENTATION AND MEASURED OUTPUT

CODE

```
for attempt in range(1, policy.max_attempts + 1):
    try: return RetryResult(value=operation(), attempts=attempt)
    except policy.retry_on as exc:
        if attempt >= policy.max_attempts or elapsed >= policy.timeout_seconds: break
        delay = min(base_delay * 2 ** (attempt - 1), max_delay)
        sleep(min(delay, remaining_timeout))
```

POLICY

max_attempts=3; timeout=100ms; delays=1ms, 2ms; cap=4ms; VERIFY_SEED=263539

OUTPUT

150 total calls (50 each at 0%, 20%, 50%)
Success rates: 100%, 98%, 92%
Observed: immediate success; failure|success; failure|failure|failure

Three attempts, 100 ms timeout, exponential delay, and observed success/exhaustion outcomes.

19. Fault-injection metrics and representative rows

The experiment used VERIFY_SEED 263539 and exactly 150 calls: 50 each at 0%, 20%, and 50% injected failure.

Injected failure	Calls	Success	Mean latency	p99 latency
0%	50	100%	0.000 ms	0.001 ms
20%	50	98%	0.276 ms	3.715 ms
50%	50	92%	0.882 ms	3.791 ms

Policy: at most three attempts, 100 ms total timeout, and bounded exponential delays of 1 ms then 2 ms (4 ms cap). Normal calls have negligible delay. At 50% injected failure, 92% still completed, while the remaining failures returned a clean error after exhausting the fixed attempt budget. A batch workload could tolerate more attempts and jitter; the interactive assistant favors a short, predictable response ceiling.

```
< .venv/bin/python scripts/run_hw05_fault_experiment.py
Vedant Vartak - DATA 260 Homework 5 fault-injection experiment
VERIFY_SEED=263539; calls=150
{"calls": 50, "injected_failure_rate": 0.0, "mean_latency_ms": 0.0, "p99_latency_ms": 0.001, "success_rate": 1.0}
{"calls": 50, "injected_failure_rate": 0.2, "mean_latency_ms": 0.276, "p99_latency_ms": 3.715, "success_rate": 0.98}
{"calls": 50, "injected_failure_rate": 0.5, "mean_latency_ms": 0.882, "p99_latency_ms": 3.791, "success_rate": 0.92}
raw_csv=/Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/fault_injection_calls.csv
<"EXHAUSTED RETRIES:",next(x for x in r if x["success"]=="False"))'
Vedant Vartak - Retry behavior evidence
IMMEDIATE SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.0', 'call_index': '1', 'success': 'True', 'attempts': '1', 'attempt_outcomes': 'success', 'latency_ms': '0.002', 'error': ''}
RETRY SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '13', 'success': 'True', 'attempts': '2', 'attempt_outcomes': 'failure|success', 'latency_ms': '1.266', 'error': ''}
EXHAUSTED RETRIES: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '46', 'success': 'False', 'attempts': '3', 'attempt_outcomes': 'failure|failure|failure', 'latency_ms': '3.773', 'error': 'operation failed after 3 attempts: injected transient storage failure'}
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Representative rows show immediate success, failure-then-success, and three exhausted attempts; the summary above is derived from all 150 CSV rows.

20. execute_tool and offline assertions

Implementation and representative assertions are immediately followed by the full named PASS output.

SAFE execute_tool + OFFLINE ASSERTIONS

execute_tool CODE

```
if name == 'search' and isinstance(inputs, dict):
    limit = inputs.get('limit', 5)
    if isinstance(limit, int) and limit > 10:
        return json.dumps(envelope(error='Safety rule blocked search: limit cannot exceed 10 records'))
    return json.dumps(call_domain_tool(name, inputs, repository), sort_keys=True)
```

ASSERTION EXAMPLES

```
assert result['ok'] and result['data']['count'] == 1
assert not invalid['ok'] and 'Invalid search input' in invalid['error']
assert not blocked['ok'] and blocked['error'].startswith('Safety rule blocked')
assert run.stop_reason == 'max_steps' and run.step_count == 3
```

OUTPUT

```
PASS search valid/invalid; detail valid/invalid; aggregate valid/invalid
PASS safety allowed/blocked; PASS agent max_steps
SUMMARY 9/9 tests passed
```

Safe execution entry point, assertion examples, and 9/9 summary.

```
< .venv/bin/python scripts/run_hw05_fault_experiment.py
Vedant Vartak - DATA 260 Homework 5 fault-injection experiment
VERIFY_SEED=263539; calls=150
{"calls": 50, "injected_failure_rate": 0.0, "mean_latency_ms": 0.0, "p99_latency_ms": 0.001, "success_rate": 1.0}
{"calls": 50, "injected_failure_rate": 0.2, "mean_latency_ms": 0.276, "p99_latency_ms": 3.715, "success_rate": 0.98}
{"calls": 50, "injected_failure_rate": 0.5, "mean_latency_ms": 0.882, "p99_latency_ms": 3.791, "success_rate": 0.92}
raw_csv=/Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/fault_injection_calls.csv
<"EXHAUSTED RETRIES:",next(x for x in r if x["success"]=="False"))'
Vedant Vartak - Retry behavior evidence
IMMEDIATE SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.0', 'call_index': '1', 'success': 'True', 'attempts': '1', 'attempt_outcomes': 'success', 'latency_ms': '0.002', 'error': ''}
RETRY SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '13', 'success': 'True', 'attempts': '2', 'attempt_outcomes': 'failure|success', 'latency_ms': '1.266', 'error': ''}
EXHAUSTED RETRIES: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '46', 'success': 'False', 'attempts': '3', 'attempt_outcomes': 'failure|failure|failure', 'latency_ms': '3.773', 'error': 'operation failed after 3 attempts: injected transient storage failure'}
< clear; .venv/bin/python scripts/test_hw05_tools.py
Vedant Vartak - DATA 260 Homework 5 offline tests
PASS test_search_valid
PASS test_search_invalid
PASS test_detail_valid
PASS test_detail_invalid
PASS test_aggregate_valid
PASS test_aggregate_invalid
PASS test_safety_rule_allowed
PASS test_safety_rule_blocked
PASS test_agent_max_steps
SUMMARY 9/9 tests passed
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Terminal output names every valid/invalid, safety, and max-step test as PASS.

21. Bounded agent loop and Ollama scenarios

The agent loop code and corresponding four-scenario terminal output are shown together.

BOUNDED AGENT LOOP CODE AND OUTPUT

CODE

```
for step in range(1, max_steps + 1):
    action = active_model.next_action(messages)
    if 'final' in action: stop_reason = 'normal_completion'; break
    result = execute_tool(action['tool'], action['inputs'], repository)
    if not parsed_result['ok'] and error.startswith('Safety rule blocked'):
        stop_reason = 'safety_rule_block'; break
    else:
        stop_reason = 'max_steps'
```

OUTPUT

```
Scenario 1: steps=2, tool_calls=1, normal_completion
Scenario 2: steps=2, tool_calls=1, normal_completion
Scenario 3: steps=2, tool_calls=1, normal_completion
Scenario 4: steps=1, tool_calls=1, safety_rule_block
```

MAX_STEPS loop, execute_tool call, normal completion, and deterministic safety stop.

```
Vedant Vartak - DATA 260 Homework 5 agent scenarios
MODEL=qwen3:8b MAX_STEPS=6 SCENARIOS=4
Scenario 1 | steps 2 | tool_calls 1 | normal_completion
  Three shrimp recalls were found: 1. **Grocery Safety Manufacturer 10** (Produce) - Frozen Shrimp Lot 00001 (REC-353
9-00001), 6,535 units affected. 2. **Grocery Safety Manufacturer 09** (Dairy and Refrigerated) - Frozen Shrimp Lot 00
004 (REC-3539-00004), 13,996 units affected. 3.
Scenario 2 | steps 2 | tool_calls 1 | normal_completion
  {'product': 'Frozen Shrimp Lot 00001', 'manufacturer': 'Grocery Safety Manufacturer 10 (Safety Contact 10)'}
Scenario 3 | steps 2 | tool_calls 1 | normal_completion
  Recalls grouped by category (min_units=0): Packaged Foods had the highest total units (1,389,038) and 131 recalls.
Meat and Seafood followed with 130 recalls (1,327,145 units). Dairy and Refrigerated (127 recalls, 1,198,565 units) a
nd Produce (112 recalls, 1,098,442 units) comple
Scenario 4 | steps 1 | tool_calls 1 | safety_rule_block
  Safety rule blocked search: limit cannot exceed 10 records
PASS: 3 normal completions + 1 safety-rule block
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Three grounded completions plus one safety-rule block using qwen3:8b.

22. Agent reflection

Selected run: Scenario 4 - deterministic safety-rule block.

I selected the fourth Ollama run because it demonstrates that the harness, not the language model, owns the safety boundary. The user prompt explicitly asked the agent to retrieve recall records with a limit of 25 and told it not to reduce that limit. In step 1, local qwen3:8b produced a structured action for the search tool with query set to an empty string and limit set to 25. The harness did not send that action directly to the database. Instead, `run_agent` routed it through the single `execute_tool(name, inputs)` entry point used by every agent tool call.

Inside `execute_tool`, the domain-specific safety rule checked the requested search limit before normal schema validation or repository access. Because the limit exceeded the allowed agent maximum of 10, the function returned the shared JSON envelope with `ok=false`, `data=null`, and the error 'Safety rule blocked search: limit cannot exceed 10 records.' It raised no exception and issued no SQL query.

The loop parsed this clean result, logged the tool name, input, complete result, step number, and run identifier to `agent_runs.jsonl`, and recognized the safety-error prefix. It then stopped immediately with `stop_reason` equal to `safety_rule_block`. The run therefore used one step and one tool-call attempt, never approached the six-step ceiling, and never gave the model a chance to bypass the policy. This run shows why centralized enforcement is stronger than asking the model to follow a prompt: even when the model follows the user's unsafe request exactly, deterministic application code blocks it consistently.

Word count: 264

23. Tool contracts and AI-use disclosure

Tool	Expected JSON	Rejected JSON	Complete error is shown on
search	{"query": str, "limit": int}	{"query": "s", "limit": 5}	Page 15
detail	{"recall_id": positive int}	{"recall_id": 0}	Page 16
aggregate	{"group_by": enum, "min_units": int}	{"group_by": "submitter", "min_units": 10}	Page 17

Pages 15-17 contain the full expected schema, valid input/output, rejected object, untruncated returned error, and explanation for each tool. All three return {ok, data, error}. Pydantic validation is caught at the boundary, and execute_tool applies the narrower agent-only limit of 10.

AI-use disclosure

1. I used an AI assistant to translate the assignment checklist into a cumulative implementation plan, draft FastAPI/Redux/MCP code, and create repeatable validation scripts. I independently reviewed the requirements, ran the tests, inspected the artifacts, and captured the final evidence.
2. I independently found that my first Home-page implementation deleted a recall directly from the record card, bypassing the required delete-confirmation UI even though the API operation worked.
3. I detected it by tracing the Home button to its Redux dispatch and reproducing the rendered flow: clicking Delete immediately issued the request instead of first showing the selected record and confirmation action.
4. I changed Home to navigate to /delete/:id, added a confirmation page that loads the selected record, and dispatches deleteRecall only after explicit confirmation. I verified the confirmation screen, success notice, and MySQL removal.

24. Submission file alignment

Required item	Repository location
Final report	reports/hw05/report.pdf
Run log	reports/hw05/RUN_LOG.txt
Metrics	reports/hw05/METRICS.md
AI disclosure	reports/hw05/AI_USE.md
Verification	reports/hw05/verification.json
Raw evidence	reports/hw05/raw/
Screenshots	reports/hw05/evidence/
Postman files	reports/hw05/postman/
Canvas upload copy	output/pdf/Vartak_HW5.pdf

The repository deliberately excludes temporary demonstration rows: UI record 6006 and Postman recall 6007/manufacture 28 were deleted after proof was captured. The final live database therefore matches the deterministic cumulative seed.

End of report